Relatório de Atividade: Aplicação Web em Containers

Ambiente: Killercoda (Ubuntu Playground)
Data de Execução: 16 de Abril de 2026
Disciplina: Administração e Serviços de Redes
Prof: Taveira
Aluno: Francisco Lucas da Silva Guedes

---

1. Resumo do Tutorial

Este relatório documenta a configuração de uma aplicação web containerizada utilizando Python (Flask) e um banco de dados relacional PostgreSQL, orquestrados via Docker Compose. O ambiente utilizado foi o playground interativo Killercoda.

Estrutura do Projeto:

- app.py: Backend em Flask com rotas de API e lógica de conexão ao banco.
- init.sql: Script de inicialização para criação automática de tabelas e carga de dados.
- requirements.txt: Arquivo de dependências contendo Flask e psycopg2-binary.
- Dockerfile: Definição da imagem para o container da aplicação Python.
- docker-compose.yml: Configuração de rede, volumes e dependência entre serviços.

2. Procedimentos Realizados

Comandos principais executados no terminal:

```
mkdir ~/python-postgres-app && cd ~/python-postgres-app
docker-compose up -d
docker-compose ps
```

Evidência Killercoda e Web

```
Editor    Tab 1    +

{
    "database": "connected",
    "status": "healthy"
}


3️⃣ Testando listagem de usuários (/users)...
[
    {
        "id": 1,
        "name": "Alice Killercoda"
    },
    {
        "id": 2,
        "name": "Bob Docker"
    },
    {
        "id": 3,
        "name": "Charlie Container"
    },
    {
        "id": 4,
        "name": "Diana DevOps"
    }
]


4️⃣ Verificando containers em execução...
List containers.

Usage: ps [options] [--] [SERVICE...]

Options:
    -q, --quiet          Only display IDs
    --services           Display services
    --filter KEY=VAL     Filter services by a property
    -a, --all            Show all stopped containers (including those created by the run command)


✅ Testes concluídos!
Attaching to flask-api
flask-api      |  * Serving Flask app 'app'
flask-api      |  * Debug mode: off
flask-api      | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
flask-api      |  * Running on all addresses (0.0.0.0)
flask-api      |  * Running on http://127.0.0.1:8080
flask-api      |  * Running on http://172.18.0.3:8080
flask-api      | Press CTRL+C to quit
flask-api      | 172.18.0.1 - - [17/Apr/2026 00:29:56] "GET / HTTP/1.1" 200 -
flask-api      | 172.18.0.1 - - [17/Apr/2026 00:29:56] "GET /health HTTP/1.1" 200 -
flask-api      | 172.18.0.1 - - [17/Apr/2026 00:29:57] "GET /users HTTP/1.1" 200 -
root@ubuntu:~/python-postgres-app$
```

f8bfa886e1be-10-244-6-118-8080.saci.r.killercoda.com

API Python + PostgreSQL

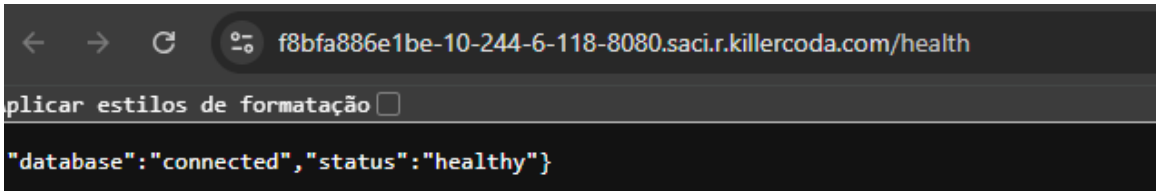
Endpoints disponíveis:

- /users - Listar usuários
- /health - Health check

3. Evidências de Funcionamento

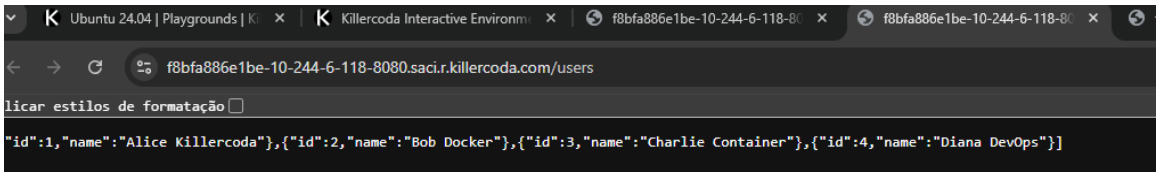
A. Health Check (Status do Banco)

A aplicação validou a conectividade com o PostgreSQL através do endpoint /health. O resultado obtido confirmou que o banco estava conectado e o status saudável.
Evidência Health



B. Listagem de Usuários (/users)

O endpoint /users retornou corretamente a lista de usuários (Alice, Bob, etc.) armazenados no PostgreSQL.



4. Tabela de Referência de Gerenciamento

Comando	Descrição da Função
docker-compose ps	Lista os containers e seus estados atuais
docker-compose logs -f	Exibe logs em tempo real para depuração
docker-compose down -v	Encerra os serviços e remove volumes de dados